

Chapter 9

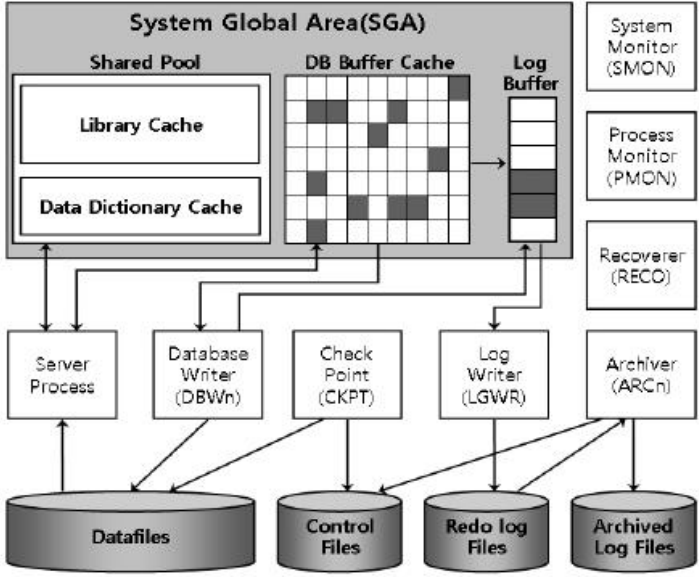
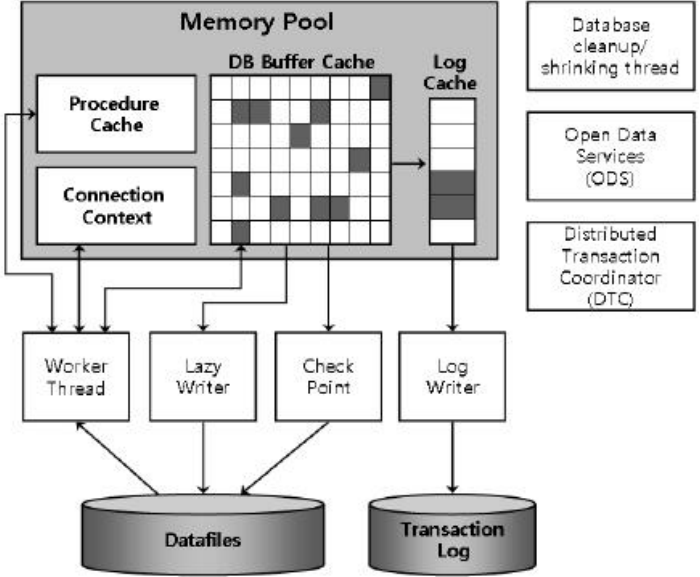
SQL 수행 구조

Outline

- 9.1 데이터베이스 아키텍처
- 9.2 SQL 처리 과정
- 9.3 데이터베이스 I/O 메커니즘

9.1 데이터베이스 아키텍처

아키텍처 모델링의 정의

오라클 아키텍처	SQL Server 아키텍처
 The Oracle architecture diagram shows the System Global Area (SGA) at the top, containing the Shared Pool (with Library and Data Dictionary Caches), the DB Buffer Cache (a grid), and the Log Buffer. To the right are background processes: SMON, PMON, RECO, and ARCH. Below the SGA are the Server Process, Database Writer (DBWn), Check Point (CKPT), Log Writer (LGWR), and Archiver (ARCn). At the bottom are the physical files: Datafiles, Control Files, Redo log Files, and Archived Log Files. Arrows indicate data flow between these components.	 The SQL Server architecture diagram shows the Memory Pool at the top, containing the Procedure Cache, Connection Context, DB Buffer Cache (a grid), and Log Cache. To the right are background services: Database cleanup/shrinking thread, Open Data Services (ODS), and Distributed Transaction Coordinator (DTC). Below the Memory Pool are the Worker Thread, Lazy Writer, Check Point, and Log Writer. At the bottom are the physical files: Datafiles and Transaction Log. Arrows indicate data flow between these components.
- 디스크에 저장된 데이터 집합(Datafile, Redo Log File, Control File 등)을 데이터베이스라 칭 - SGA 공유 메모리 영역과 이를 액세스하는 프로세스 집합을 합쳐서 인스턴스(Instance)	master, model, msdb, tempdb 등의 시스템 데이터베이스가 만들어지며, 여기에 사용자 데이터베이스를 추가로 생성하는 구조

9.1 데이터베이스 아키텍처

프로세스

서버프로세스와 백그라운드 프로세스의 집합으로 서버프로세스는 사용자로부터 전달받은 각종 명령을 처리하고, 백그라운드 프로세스는 할당을 받아 역할을 백그라운드에서 수행

프로세스 - 서버 프로세스(Server Processes)

사용자 프로세스와 통신하면서 각종 명령을 처리한다.

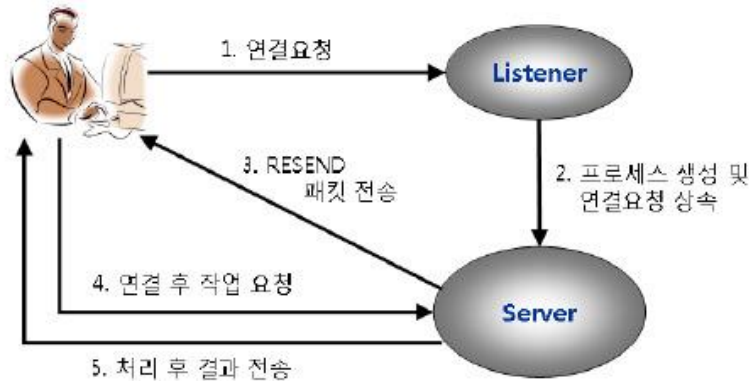
※ SQL Server에선 Worker 스레드가 같은 역할을 담당

SQL 파싱, 최적화를 수행, 커서를 열어 SQL을 실행하면서 블록을 읽어 클라이언트로 요청결과를 전송

9.1 데이터베이스 아키텍처

프로세스 - 서버 프로세스(Server Processes)

전용
서버
방식



- 사용자 별 프로세스를 할당, 예를 들어 50개의 서버 프로세스와 연결된 50개 사용자 프로세스를 공유해 반복 재사용하는 방식

공유
서버
방식



- 하나의 서버 프로세스를 여러 사용자 세션이 공유하는 방식
- Connection Pooling 기법을 DBMS 내부에 구현해 놓은 방식

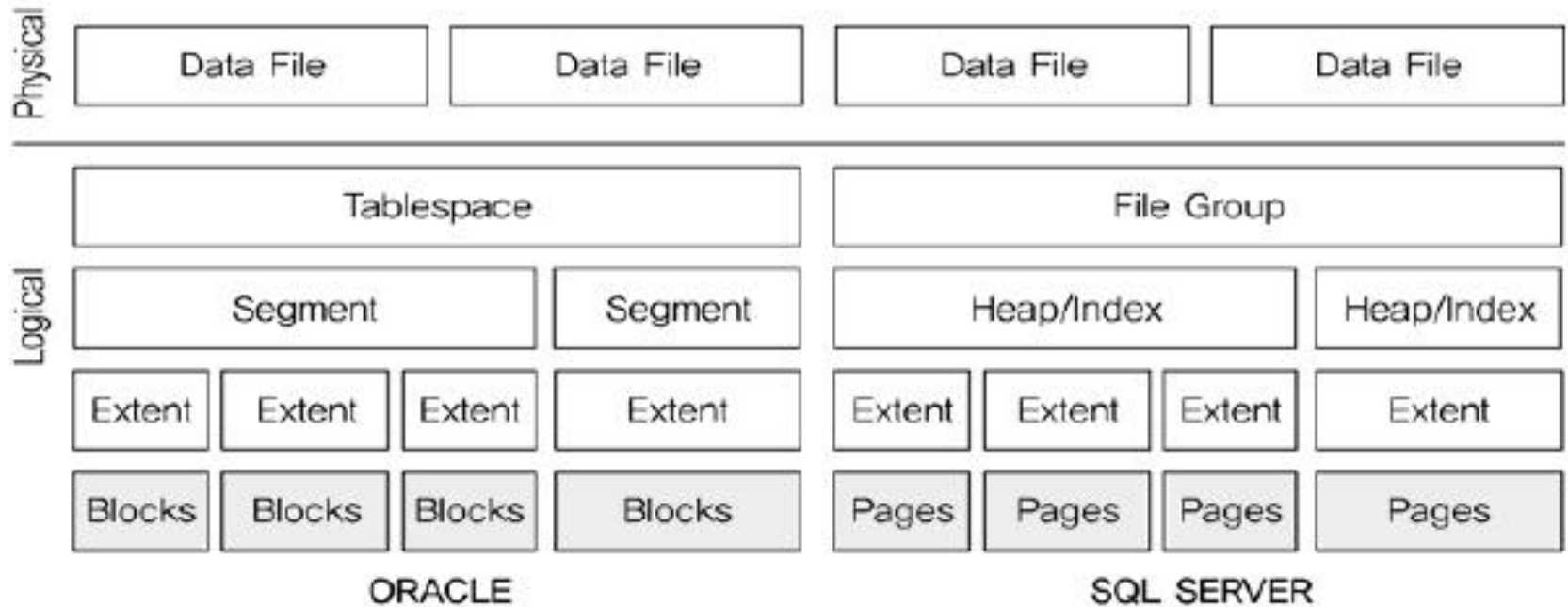
9.1 데이터베이스 아키텍처

프로세스 - 백그라운드 프로세스(Background Processes)

ORACLE	SQL SERVER	설명
System Monitor(SMON)	Database cleanup / shrinking thread	장애가 발생한 시스템을 재기동할 때 인스턴스 복구를 수행하고, 임시 세그먼트와 익스텐트를 모니터링한다.
Process Monitor(PMON)	Open Data Services(OPS)	이상이 생긴 프로세스가 사용하던 리소스를 복구한다.
Database Writers(DBWn)	Lazywriter thread	버퍼 캐시에 있는 Dirty 버퍼를 데이터 파일에 기록한다.
Log Writer(LGWR)	Log writer thread	로그 버퍼 엔트리를 Redo 로그 파일에 기록한다.
Archiver(ARCn)	N/A	꼭 찬 Redo 로그가 덮어 쓰여지기 전에 Archive 로그 디렉토리로 백업한다.
Checkpoint (CKPT)	Database Checkpoint thread	Checkpoint 프로세스는 이전에 Checkpoint가 일어났던 마지막 시점 이후의 데이터베이스 변경 사항을 데이터 파일에 기록하도록 트리거링하고, 기록이 완료되면 현재 어디까지 기록했는지를 컨트롤 파일과 데이터 파일헤더에 저장한다. 좀 더 자세히 설명하면, Write Ahead Logging 방식(데이터 변경 전에 로그부터 남기는 메커니즘)을 사용하는 DBMS는 Redo 로그에 기록해 둔 버퍼 블록에 대한 변경사항 중 현재 어디까지를 데이터 파일에 기록했는지 체크포인트 정보를 관리해야 한다. 이는 버퍼 캐시와 데이터 파일이 동기화된 시점을 가리키며, 장애가 발생하면 마지막 체크포인트 이후 로그 데이터만 디스크에 기록함으로써 인스턴스를 복구할 수 있도록 하는 용도로 사용된다. 이 정보를 갱신하는 주기가 길수록 장애 발생 시 인스턴스 복구 시간도 길어진다.
Recoverer(RECO)	Distributed Transaction Coordinator(DTC)	분산 트랜잭션 과정에 발생한 문제를 해결한다.

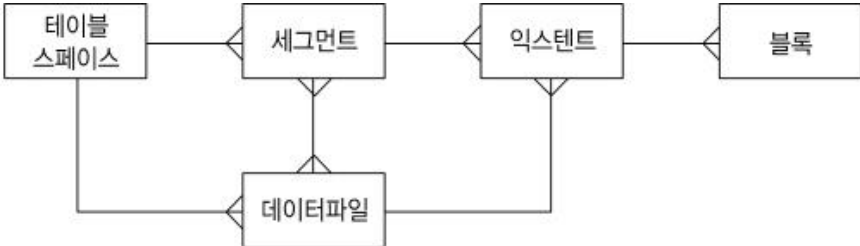
9.1 데이터베이스 아키텍처

파일 구조 - 데이터 파일



9.1 데이터베이스 아키텍처

파일 구조 - 데이터 파일

구분	설명
블록 / 페이지	- 데이터를 읽고 쓸 때의 논리적인 단위 - DB의 성능지표는 액세스 블록 개수로 옵티마이저가 실행계획 수립 시 판단에 영향을 미친다.
익스텐트	- 테이블 스페이스로부터 공간을 할당하는 단위 - 데이터 입력 시 공간이 부족해지면 익스텐트 크기의 연속된 블록으로 할당된다. - 익스텐트 내 블록은 논리적으로 인접하지만, 물리적으로 익스텐트끼리 서로 인접하지 않는다. - 유형 : 균일(Uniform) 익스텐트, 혼합(Mixed) 익스텐트
세그먼트	- 테이블, 인덱스, Undo처럼 저장공간을 필요로 하는 데이터베이스 오브젝트 - 세그먼트는 자신이 속한 테이블 스페이스 내 여러 데이터 파일에 걸쳐 저장을 해 디스크 경합을 줄이고 I/O 분산 효과를 만든다. - 테이블을 생성할 때, 내부적으로는 테이블 세그먼트가 생성 - 인덱스를 생성할 때, 내부적으로 인덱스 세그먼트가 생성
테이블 스페이스	- 세그먼트의 집합으로 여러 데이터 파일로 구성  <pre>graph TD; TS[테이블 스페이스] --> S[세그먼트]; S --> E[익스텐트]; E --> B[블록]; S --> DP[데이터파일]; E --> DP; TS --> DP;</pre>

9.1 데이터베이스 아키텍처

파일 구조 - 그 외 파일

구분	설명
임시 데이터 파일	- 대량의 정렬이나 해시 작업을 수행하다가 메모리 공간이 부족해지면 중간 결과집합을 저장하는 용도의 파일
Online Redo 로그 (Oracle)	- 체크포인트 이후 발생하는 모든 트랜잭션을 기록하는 로그파일 - 인스턴스가 비정상 종료가 되면 복구를 위해 사용됨 - 두개 이상의 파일로 구성되며 라운드 로빈 방식으로 파일이 짝 차면 다음 파일로 이동해 기록
트랜잭션 로그 (SQL Server)	- 데이터베이스마다 트랜잭션 로그 파일이 하나씩 생긴다. - 트랜잭션 로그 파일은 내부적으로 작은 단위의 세그먼트로 나뉘며, 이 개수가 많아지지 않도록 옵션을 지정해야 한다. (조각화 문제)
Archived Redo 로그 (Offline)	- Online Redo 로그가 재사용되기 전에 다른 위치로 백업해 둔 파일

9.1 데이터베이스 아키텍처

메모리 구조

구분	설명
시스템 공유 메모리 영역	- 여러 프로세스(또는 스레드)가 동시에 액세스할 수 있는 메모리 영역 - Oracle : 'System Global Area(SGA)', SQL Server : 'Memory Pool' - 구성되는 캐시 영역 : DB 버퍼 캐시, 공유 풀, 로그 버퍼 - 액세스 직렬화 메커니즘 사용 [래치(Latch), 버퍼 Lock, 라이브러리 캐시 Lock/Pin]
프로세스 전용 메모리 영역	- 오라클은 서버 프로세스가 전용 메모리 영역을 가질 수 있는데, 이를 'Process Global Area(PGA)'라고 부르며, 데이터를 정렬하고 세션과 커서에 관한 상태 정보를 저장하는 용도로 사용 - SQL Server는 프로세스 전용 메모리를 가지지 않음

9.1 데이터베이스 아키텍처

메모리 구조 - 시스템 공유 메모리 영역의 구성요소

구분	설명
DB 버퍼 캐시 (DB Buffer Cache)	- 데이터 파일로부터 읽어 들인 데이터 블록을 담는 캐시 영역 - 버퍼블록 상태 > Free 버퍼 : 비어있는 상태 > Dirty 버퍼 : 버퍼에 캐시된 후 변경되었지만 디스크에 기록되지 않은 상태 > Pinned 버퍼 : 읽기 또는 쓰기 작업이 진행중인 상태 - LRU(least recently used) 알고리즘을 사용해 사용빈도가 높은 블록을 위주로 구성
공유 풀 (Shared Pool)	- 딕셔너리 캐시 : 테이블, 인덱스 같은 오브젝트는 물론 테이블 스페이스, 데이터 파일, 세그먼트, 익스텐트, 사용자, 제약에 관한 메타 정보를 저장하는 캐시 - 라이브러리 캐시 : 사용자가 수행한 SQL문과 실행계획, 저장 프로시저를 저장해 두는 캐시
로그 버퍼 (Log Buffer)	- 로그파일에 기록하기전에 저장하는 버퍼, 일정량을 모아서 로그파일에 기록 - 로그버퍼에 있는 변경사항은 적어도 Commit 시점에는 로그파일에 기록 - 로그버퍼는 유실 가능성이 있기 때문에 로그파일에 기록된 내용으로 트랜잭션을 복원
PGA (Process Global Area)	- User Global Area(UGA) : 각 세션을 위한 독립적인 메모리 공간 - Call Global Area(CGA) : Call이 진행되는 동안 저장되는 메모리 영역 - Sort Area : 데이터 정렬을 위해 사용

9.1 데이터베이스 아키텍처

대기 이벤트

DBMS 내부에선 많은 프로세스 간에는 상호작용이 필요해 다른 프로세스가 종료될 때까지 대기 해야 하는 상황이 자주 발생

$$\begin{aligned}\text{Response Time} &= \text{Service Time} + \text{Wait Time} \\ &= \text{CPU Time} + \text{Queue Time}\end{aligned}$$

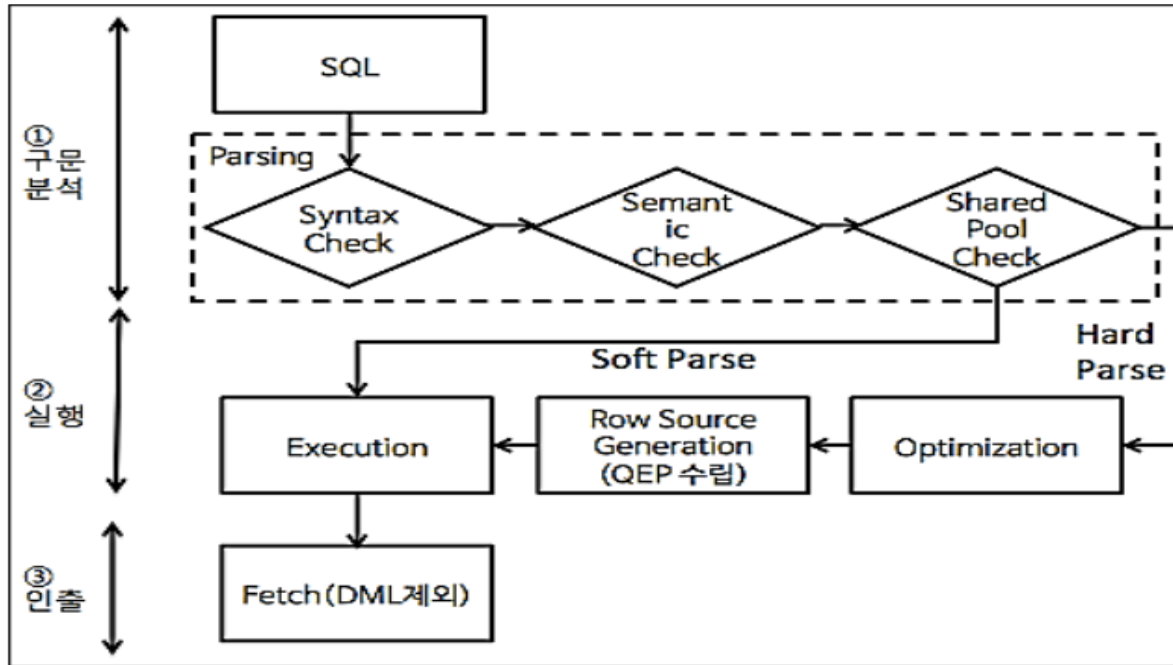
구분	설명
응답 시간(Response Time)	- 프로세스의 실행결과가 응답이 올 때까지 시간
서비스 시간(Service Time)	- 프로세스가 정상적으로 동작하며 일을 수행한 시간
대기 시간(Wait Time)	- 프로세스가 잠시 수행을 멈추고 대기한 시간

9.1 데이터베이스 아키텍처

대기 이벤트

대기발생지점	설명
라이브러리 캐시 부하	- 라이브러리 캐시에서 SQL 커서를 찾고 최적화하는 과정에서 경합이 발생해 대기 이벤트발생 - 수행 중인 SQL이 참조하는 오브젝트를 다른 사용자가 DDL 문장을 수행할 때 발생 - library cache lock, library cache pin
데이터베이스 Call과 네트워크 부하	- 애플리케이션과 네트워크 구간에서 소모된 시간 - SQL*Net message from client, SQL*Net message to client - SQL*Net more data to client, SQL*Net more data from client
디스크 I/O 부하	- db file sequential read, db file scattered read, direct path read, direct path write - direct path write temp, direct path read temp, db file parallel read
버퍼 캐시 경합	- 버퍼 캐시에서 블록을 읽는 과정에 경합이 발생할 경우 발생하는 대기이벤트 - latch: cache buffers chains, latch: cache buffers lru chain - buffer busy waits, dfree buffer waits
Lock 관련 대기 이벤트	- 'enq'로 시작되는 대기 이벤트는 Lock과 관련된 대기이벤트 - enq: TM – contention, enq: TX - row lock contention, enq: TX - index contention - enq: TX - allocate ITL entry, enq: TX contention, latch free

9.2 SQL 처리 과정



처리과정	상세 처리과정	설명	비고
구문분석 (Parsing)	Syntax Check	- SQL문장의 구문 분석	Prepared statement에서는 생략
	Semantic Check	- SQL문장의 의미 및 문법적 절적성 체크	
	Shared Pool Check	- 쿼리에 사용된 테이블, 컬럼, 뷰 등 존재 여부확인	
	Optimization	- 쿼리 수행을 위한 최적의 실행방법 선택(RBO/CBO)	
	Execution Plan	- 쿼리 실행 계획 수립	
실행 (Execution)	캐시데이터 검색	- 메모리영역의 데이터베이스 캐시영역을 검색하여 해당 객체가 존재하는지 확인	Cache Database file
	데이터파일 검색	- 캐시 내에 존재하지 않는 경우, 데이터베이스 파일을 검색하여 해당 데이터를 캐시 영역에 저장	
인출(Fetch)	데이터 인출	- 관련 테이블 데이터를 처리하여 사용자에게 전송	DML생략

9.3 데이터베이스 I/O 메커니즘

블록 단위 I/O

DBMS에서는 I/O는 블록(SQL Server는 페이지) 단위로 이루어짐
하나의 레코드를 읽더라도 레코드가 있는 블록 전체를 읽어서
SQL 성능에 대한 중요한 지표로 사용되며 옵티마이저가 판단
할때 큰 영향을 미치는 지표
블록 단위 I/O는 버퍼 캐시, 데이터파일 I/O 모두 적용

- 데이터 파일에서 DB 버퍼 캐시로 블록을 적재할 때

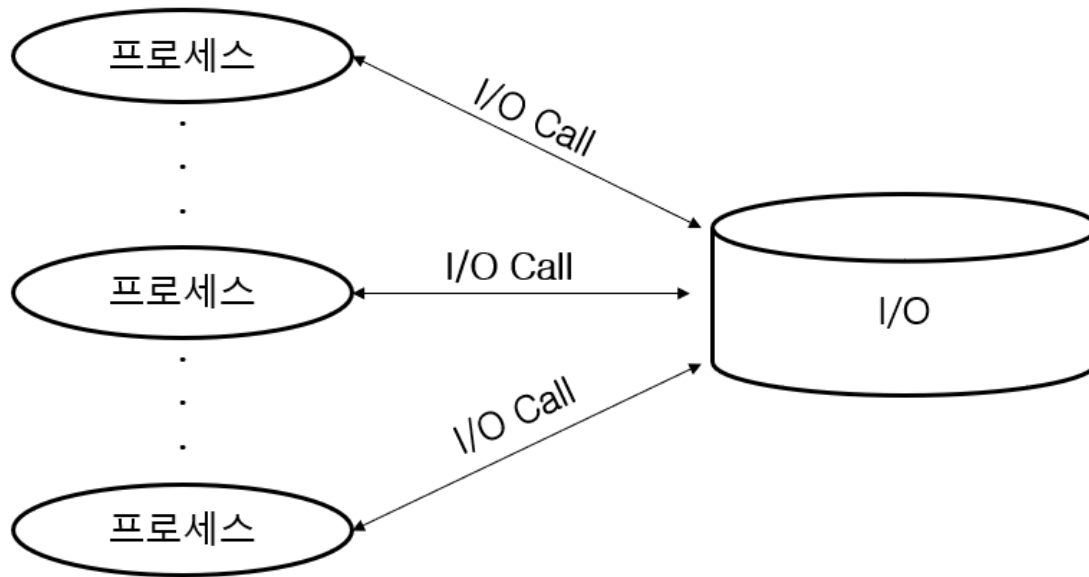
- 데이터 파일에서 블록을 직접 읽고 쓸 때

- 버퍼 캐시에서 블록을 읽고 쓸 때

- 버퍼 캐시에서 변경된 블록을 다시 데이터 파일에 쓸 때

9.3 데이터베이스 I/O 메커니즘

I/O 효율화 튜닝의 중요성



디스크 I/O를
최소화하고 버퍼 캐시
효율을 높이는 것이
데이터베이스 I/O
튜닝의 목표

DBMS는 읽어야 하는 블록을 버퍼 캐시에서 먼저 찾아보고, 없을 경우에는 디스크에서 읽어 버퍼 캐시에 적재 후 읽기/쓰기를 수행
디스크 I/O가 필요해지만 서버 프로세스는 시스템에 I/O Call을
하고 잠시 대기 상태에 들어 간다.

9.3 데이터베이스 I/O 메커니즘

버퍼 캐시 히트율(BCHR, Buffer Cache Hit Ratio)

전체 읽은 블록 중에서 메모리 버퍼 캐시에서 찾은 비율

$$BCHR = (\text{버퍼 캐시에서 곧바로 찾은 블록 수} / \text{총 읽은 블록 수}) \times 100$$

BCHR이 높으면 **디스크 I/O가 줄어드는 효과가 발생**한다.

다만, BCHR의 성능지표의 한계점은 같은 블록을 반복적으로 액세스하는 형태의 SQL은 논리적인 I/O 요청이 비효율적으로 많이 발생함에도 불구하고 BCHR은 매우 높게 나타난다.

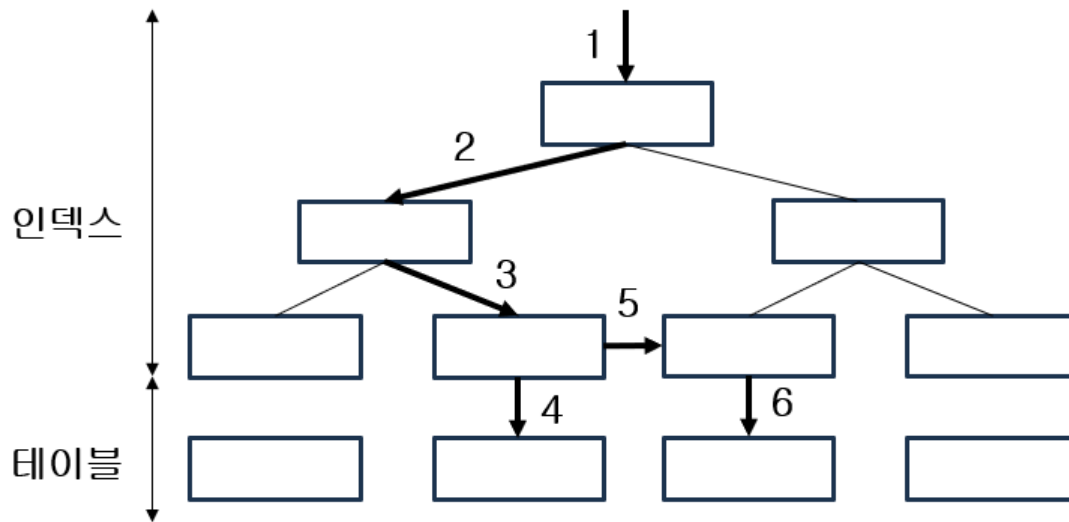
논리적 I/O가 많아지면 블록을 찾는 과정에서 경합 발생 시 메모리 I/O와 디스크 I/O보다 비용이 커질 수 있어 **논리적 블록 읽기도 최소화** 해야 한다.

네트워크, 파일시스템 캐시가 I/O 효율에 미치는 영향

네트워크에 전송되는 데이터 량도 I/O 속도에 영향을 미친다.
SQL 작성 시 네트워크 전송량을 줄일 수 있도록 처리가 필요

9.3 데이터베이스 I/O 메커니즘

Sequential 액세스와 Random 액세스



구분	설명
Sequential 액세스	- 레코드간 논리적 또는 물리적인 순서를 따라 차례대로 읽어 나가는 방식 - 5번 처럼 이동하는 방식
Random 액세스	- 레코드간 논리적, 물리적인 순서를 따르지 않고, 한 건을 읽기 위해 한 블록 씩 접근하는 방식 - 1, 2, 3, 4, 6 번 순으로 이동하는 방식

Sequential 액세스에 의한 선택 비중을 높이면서 Random 액세스를 최소화 하는 것이 중요하다.

9.3 데이터베이스 I/O 메커니즘

Sequential 액세스와 Random 액세스

구분	SQL
테이블	<pre>CREATE TABLE TBL_TEMP (ID NUMBER PRIMARYKEY ,NAME VARCHAR2(10) ,JOB VARCHAR2(10) ,DEPT VARCHAR2(10))</pre>
인덱스	<pre>CREATE INDEX IDX_TEMP ON TBL_TEMP(ID, NAME)</pre>
조회 SQL1	<pre>SELECT ID , NAME FROM TBL_TEMP WHERE ID = '10'</pre>
조회 SQL2	<pre>SELECT ID , NAME , JOB , DEPT FROM TBL_TEMP WHERE ID = '10'</pre>
조회 SQL3	<pre>SELECT ID , NAME FROM TBL_TEMP WHERE DEPT = '10'</pre>

9.3 데이터베이스 I/O 메커니즘

Sequential 액세스와 Random 액세스

조회 SQL1	<pre>SELECT ID , NAME FROM TBL_TEMP WHERE ID = '10'</pre>	인덱스의 컬럼만 사용 -> 가장 효율 좋음
조회 SQL2	<pre>SELECT ID , NAME , JOB , DEPT FROM TBL_TEMP WHERE ID = '10'</pre>	인덱스 이외의 컬럼 사용 -> 인덱스 스캔 후 결과집합만 테이블 블록 스캔
조회 SQL3	<pre>SELECT ID , NAME FROM TBL_TEMP WHERE DEPT = '10'</pre>	인덱스 이외의 컬럼 사용 -> 조회 시 테이블 블록을 스캔 -> Random 액세스 증가

9.3 데이터베이스 I/O 메커니즘

Single Block I/O와 MultiBlock I/O

구분	Single Block I/O	MultiBlock I/O
개념	- 한번의 I/O Call에 하나의 데이터 블록만 읽어 메모리에 적재 하는 방식	- I/O Call이 인접한 블록들을 같이 읽어 메모리에 적재 하는 방식
스캔	- 인덱스 스캔 - 테이블 블록 스캔	- Table Full Scan 처럼 저장된 순서에 따라 읽는 방식에 유리
I/O 횟수	- MultiBlock I/O 보다 많음	- 인접 블록을 같이 읽어 I/O 횟수가 적음
메모리 유지시간	- 조회한 블록은 무조건 사용하기 때문에 메모리상 오래 유지됨 (LRU 기준 시작 쪽에 위치)	- 조회한 블록이 사용되지 않기 때문에 메모리 상 오래 유지 되지 않음 (LRU 기준 끝 쪽에 위치)

9.3 데이터베이스 I/O 메커니즘

I/O 효율화 원리

원리	설명
필요한 최소 블록만 읽도록 SQL 작성	- 동일한 데이터를 중복 액세스하지 않고, 필요한 컬럼만 조회하는 형태로 SQL 집합정의
최적의 옵티마이징 팩터 제공	- 인덱스를 효율적으로 구성 - DBMS 기능(파티션, 클러스터, 윈도우 함수) 활용해 옵티마이저의 최적 선택을 유도 - 옵티마이저 모드(전체 처리속도 최적화, 최초 응답속도 최적화)와 파라미터 변경 - 통계정보 관리
옵티마이저 힌트를 사용해 최적의 액세스 경로로 유도	- 옵티마이저가 생각만큼 최적의 실행계획을 수립하지 못하는 경우 힌트로 유도